

ADDER CIRCUIT

1. HALF ADDER

A half adder adds two 1-bit inputs.

Inputs and Outputs

Signal	Type	Width	MSB	LSB
A	Input	1 bit	A	A
B	Input	1 bit	B	B
Sum	Output	1 bit	Sum	Sum
Carry	Output	1 bit	Carry	Carry

MAIN CODE

```
module half_adder (  
    input wire A,  
    input wire B,  
    output wire Sum,  
    output wire Carry  
);  
  
    assign Sum = A ^ B;  
    assign Carry = A & B;  
  
endmodule
```

TEST BENCH CODE

```
`timescale 1ns/1ps  
  
module half_adder_tb;
```

```
reg A;
```

```
reg B;
```

```
wire Sum;
```

```
wire Carry;
```

```
half_adder uut (
```

```
    .A(A),
```

```
    .B(B),
```

```
    .Sum(Sum),
```

```
    .Carry(Carry)
```

```
);
```

```
initial
```

```
begin
```

```
    A = 0; B = 0; #10;
```

```
    A = 0; B = 1; #10;
```

```
    A = 1; B = 0; #10;
```

```
    A = 1; B = 1; #10;
```

```
    $finish;
```

```
end
```

```
initial
```

```
begin
```

```
    $monitor("Time=%0t | A=%b | B=%b | Sum=%b | Carry=%b",
```

```
        $time, A, B, Sum, Carry);
```

```
end
```

```
endmodule
```

Sure. For your Full Adder using Xilinx Vivado, use this simple Verilog code.

1. Inputs and Outputs

Signal	Type	Width	MSB	LSB
A	Input	1 bit	A	A
B	Input	1 bit	B	B
Cin	Input	1 bit	Cin	Cin
Sum	Output	1 bit	Sum	Sum
Cout	Output	1 bit	Cout	Cout

MAIN CODE

```
module full_adder (  
    input wire A,  
    input wire B,  
    input wire Cin,  
    output wire Sum,  
    output wire Cout  
);  
  
    assign Sum = A ^ B ^ Cin;  
    assign Cout = (A & B) | (B & Cin) | (A & Cin);  
  
endmodule
```

TEST BENCH CODE

```
`timescale 1ns/1ps  
  
module full_adder_tb;
```

```
reg A, B, Cin;  
wire Sum, Cout;
```

```
full_adder uut (  
    .A(A),  
    .B(B),  
    .Cin(Cin),  
    .Sum(Sum),  
    .Cout(Cout)  
);
```

```
initial begin
```

```
    A=0; B=0; Cin=0; #10;  
    A=0; B=0; Cin=1; #10;  
    A=0; B=1; Cin=0; #10;  
    A=0; B=1; Cin=1; #10;  
    A=1; B=0; Cin=0; #10;  
    A=1; B=0; Cin=1; #10;  
    A=1; B=1; Cin=0; #10;  
    A=1; B=1; Cin=1; #10;
```

```
    $finish;
```

```
end
```

```
initial begin
```

```
    $monitor("Time=%0t | A=%b B=%b Cin=%b | Sum=%b Cout=%b",  
        $time, A, B, Cin, Sum, Cout);
```

```
end
```

```
endmodule
```

